CITS 3002
Computer Networks

Lab 2: Stop and wait

Note: This lab runs for 2 weeks, so don't panic if you don't finish in the first week.

Worth: 2% of the unit

Please note: ask your lab demonstrator about any questions you have regarding previous labs. It will be beneficial for you to have a good foundation early on in the semester, and we are here to help!

Stop-and-wait (SAW) protocol is a simple protocol to transfer data between network nodes, as per the description in the lectures. The simulation code (in Python) that implements SAW protocol as seen in the lecture has been provided to you (containing five Python code files `ticktock.py`, `sim.py`, `defs.py`, `checksums.py` and `stopandwait.py`, and four topology files `TICKTOCK`, `STOPANDWAIT1`, `STOPANDWAIT2`, and `STOPANDWAIT3`). You won't need to modify these files, but you should execute the protocol with each of the given topology files.

The topology file TICKTOCK provides a basic topology, you should inspect it to see what it contains. Similarly, STOPANDWAIT1 defines the basic topology consisting of just 2 nodes and 1 link - the minimum we require for a Data-Link Layer protocol. The topology file STOPANDWAIT2 introduces frame corruption and the topology file STOPANDWAIT3 introduces frame loss.

In this lab, you will be investigating the performance of SAW protocol, as well as implement some features and examine if it would improve its performance (or not).

1. Tasks

TASK 1. TICKTOCK

Examine the topology file `TICKTOCK` and its corresponding "protocol" file `ticktock.py`. Together they define a simple topology of two nodes connected by a single bidirectional link. The program in `ticktock.py` demonstrates the use of timer events which you will later use to provide timeouts in your protocols. To run, from the command-line type (you may need to provide the full path to the Python 3 executable if it is not in your PATH):

```
prompt> python sim.py TICKTOCK
```

You should see a lot of output like this:

```
[0]:  1748      tick
[1]:  1748      tick
[0]:  1749              tock
[1]:  1749              tock
```

Interrupt Python to stop the simulator (i.e. with Control+C) when you get bored.

Note that each node's output (as written with the `print()` function) is being prepended with the `nodenumber` (`[0]:` or `[1]:`). Also, we are receiving ticks and tocks at a rate much faster than one per second. This is because the simulator is processing events as fast as it possibly can, so one second of simulation time is passing much faster than one second of our time. If you run the simulator without a topology file:

```
prompt> python.exe sim.py
```

You will see that there are some command line options which can affect the simulator's behaviour. Try limiting the duration of the simulation by specifying some arguments e.g. `-e 10s`.

Although this is a rather trivial exercise, make sure that you understand what is happening. Read through the protocol code (ticktock.py) to determine what is happening (do not concern yourself with the simulator code in sim.py).

You can also ask the simulator to print statistical information to a CSV file at regular intervals (of simulation time). Try running TICKTOCK and getting some statistics with the following command line:

```
prompt> python sim.py -e 20s --stats-csv tickstats.csv TICKTOCK
```

You can change how often statistics are written to the CSV using the `--stats-period` argument. Run the TICKTOCK example for 10 seconds, with 1 sec intervals. Seed it with the value 19, and then 100.

TASK 2. STOP AND WAIT PROTOCOL – RUNNING IT

Running the simulation with different topology files is as follows (from cmd):

```
python sim.py --execution-duration 7200s --stats-csv saw1.csv --silent-nodes -S 42 STOPANDWAIT1
python sim.py --execution-duration 7200s --stats-csv saw2.csv --silent-nodes -S 42 STOPANDWAIT2
python sim.py --execution-duration 7200s --stats-csv saw3.csv --silent-nodes -S 42 STOPANDWAIT3
```

the sim.py takes several arguments, which are described as follows:

Argument command	Meaning
Execution-duration	The simulation runtime
Stats-csv	Output the statistics to this csv file
Silent-nodes	Omit terminal (cmd) outputs
S	Set random number generator seed value

So to simply display the packets between the nodes, you can execute:

```
python sim.py --execution-duration 100s -S 42 STOPANDWAIT1
```

Now, for each of the topology files, follow the path that each frame will take "through the source code" when there's no Physical Layer problems, then frame corruption, and then frame loss. Gather some basic statistics from this simulator (edit code as necessary) that show how the probabilities of frame corruption and loss affect the number of frames delivered in a fixed length of time.

CITS 3002
Computer Networks

TASK 3. VISUALISATION

We will finally investigate the performance of our stop-and-wait protocol - how it performs under varying characteristics of (random) frame corruption and loss. We will need to run a number of simulations, varying just one attribute while holding all others constant.

The simulation code provided already outputs the simulation results to csv files. Your task here is to write a code to generate a chart that displays the time vs messages delivered for the three topology files provided. Although ideally your code runs similar to the simulator (i.e., take arguments of csv files), it is okay to plot directly the three csv files generated.

TASK 4. UPGRADING STOP AND WAIT PROTOCOL

In task 1, we experimented with a complete stop-and-wait protocol which used acknowledgements to report the successful arrival of data frames. If a corrupted frame arrived, the receiver would simply ignore it and the sender's timeout mechanism would cause retransmission of the data frame.

This protocol, although robust, introduces some inefficiency because the sender is not directly (or quickly) informed about a corrupt data frame arriving at the receiver. The sender actually only infers the data frame corruption after a timeout occurs. The use of an additional negative-acknowledgement frame will reduce this inefficiency.

Your task is to modify a copy of the provided stop-and-wait protocol to also use negative-acknowledgements*. Have the receiver reply with a negative-acknowledgement when a corrupt frame arrives.

Once completed, plot the chart you implemented in task 2 to see if there has been any improvements.

*This is an interesting problem - don't panic about the apparent need for lots of tricky code. Think about the problem - it requires a bit of logic and only a few additional lines of code.

2. Quiz on LMS

To gain 2% allocated mark for this lab, please ensure that you solve lab questions on LMS by the deadline.